

µOS++ IIIe test framework — developer guide

How to **configure**, **run** and **write** tests in the xPack/xpm test framework of `micro-os-plus-iii` (branch `smp`).

This guide follows the workflow of the upstream [README-DEVELOPER.md](#): every command is an `xpm run <action>` executed in the `tests/` folder (or with `-C <repo>/tests`), and the same four verbs — `install-all`, `test`, `test-all`, `deep-clean` — are the entry points. Everything below that is the machinery those verbs drive, and how to extend it.

Every statement in this guide was checked against the files it names. When another document disagrees, the files win.

Contents

1. Who this is for
2. Prerequisites and workspace
3. Quick start (the README-DEVELOPER flow)
4. Architecture in one picture
5. Layer 1 — `tests/package.json` (xpm)
6. Layer 2 — the CMake harness (`tests/`)
7. Layer 3 — the port test builders
8. Layer 4 — CTest, runners and verdicts
9. Running tests day to day
10. Writing a new test
11. Adding a configuration or a platform
12. Checklist before you commit
13. Troubleshooting
14. Reference tables

1. Who this is for

- You want to **run** the existing tests on the host, under QEMU, or on a board → read §2, §3 and §9.
- You want to **add a test** → read §4, then §10. §5–§8 explain every file a recipe touches.
- You want to **add a board, a toolchain or a build type** → §11.

Vocabulary used throughout:

term	meaning
configuration (<code>C</code>)	one entry of <code>xpack.buildConfigurations</code> in <code>tests/package.json</code> , e.g. <code>cortexm-pico2-cmake-gcc-debug</code> . It owns one build folder, <code>tests/build/<C></code> .
platform	the value of <code>PLATFORM_NAME</code> , e.g. <code>cortexm-pico2</code> . One folder under <code>tests/platforms/</code> . Several configurations (debug/release, toolchains) share one platform.
port	an architecture repository: <code>micro-os-plus-iii-{aarch32,aarch64,cortexm,posix-arch}</code> .

term	meaning
board	the port's name for a physical/emulated board (<code>BOARD</code>), e.g. <code>pico2-rp2350b-psram</code> .
harness suite	a portable test in <code>tests/sources/<name>/</code> , entry point <code>os_main()</code> , no <code>main()</code> .
board test	a test owned by one board of one port, in <code><port>/test/<board>/<name>/</code> , with its own <code>main()</code> .
variant	how a test image runs: <code>-qemu</code> (emulator), <code>-host</code> (native process), <code>-hwd</code> (real board over OpenOCD).

2. Prerequisites and workspace

Tools

- A recent `xpm` (the harness requires `minimumXpmRequired: 0.20.8`), which is a portable [Node.js](#) command line application:

```
npm install --global xpm@latest
```

- `bash`, `git`, and for hardware runs a USB debug probe (CMSIS-DAP / ST-Link / J-Link, per board).

Everything else — CMake, Ninja, the host GCC/clang versions, `arm-none-eabi-gcc`, `aarch64-none-elf-gcc`, `qemu-arm`, `openocd` — is **pinned per configuration** in `tests/package.json` and installed by xpm into `tests/build/<C>/xpacks/.bin`. Nothing is taken from your system `PATH` except in the `native-cmake-sys-*` configurations, whose purpose is to test the system compiler.

Workspace layout

The kernel and its ports are separate repositories that must be **siblings**:

```
~/Work/micro-os-plus/
├─ micro-os-plus-iii/           the kernel + the test harness (branch smp)
│   └─ tests/                  ← every xpm command runs here
│       └─ test_smpl/          the runners: run-qemu.sh, run-host.sh, run-hw.sh
├─ micro-os-plus-iii-aarch32/  AArch32 port (Raspberry Pi, Luckfox Lyra)
├─ micro-os-plus-iii-aarch64/  AArch64 port (Raspberry Pi)
├─ micro-os-plus-iii-cortexm/   Cortex-M port (RP2350, STM32F4, QEMU cores)
├─ micro-os-plus-iii-posix-arch/ POSIX port (native host process)
└─ micro-os-plus-iii-devices/   shared device drivers
```

`tests/cmake/tests-main.cmake` finds the ports as `<tests>/../../micro-os-plus-iii-<port>`, and falls back to the old `micro-os-plus-iii-<port>.git` names when the plain name has no `CMakeLists.txt`. To use a copy somewhere else, pass the cache variable on `prepare` (see §6.3), e.g. `-D UOS_CORTEXM_DIR=/path/to/cortexm`.

To clone the workspace:

```
mkdir -p ~/Work/micro-os-plus && cd ~/Work/micro-os-plus
for r in micro-os-plus-iii micro-os-plus-iii-aarch32 micro-os-plus-iii-aarch64 \
    micro-os-plus-iii-cortexm micro-os-plus-iii-posix-arch micro-os-plus-iii-devices
do
    git clone --branch smp "<origin>/$r.git" "$r" # <origin>: your remote; devices uses master
done
```

or, to update an existing workspace:

```
for r in ~/Work/micro-os-plus/micro-os-plus-iii*; do git -C "$r" pull; done
```

3. Quick start (the README-DEVELOPER flow)

The commands below are exactly the upstream `README-DEVELOPER.md` sequence, pointed at this workspace. `-C` makes xpm run in the `tests/` folder, so they work from anywhere.

Top dependencies (the build helper, `del-cli`, ...), once per clone:

```
npm --prefix ~/Work/micro-os-plus/micro-os-plus-iii/tests install
```

Satisfy dependencies for all configurations — downloads every pinned toolchain and emulator into each configuration's build folder (large, one-off):

```
xpm run install-all -C ~/Work/micro-os-plus/micro-os-plus-iii/tests
```

Run a first test set — `test` is the quick one (the `qemu-cortex-m7f` debug and release suites):

```
xpm run test -C ~/Work/micro-os-plus/micro-os-plus-iii/tests
```

Run all tests with all available toolchains:

```
xpm run test-all -C ~/Work/micro-os-plus/micro-os-plus-iii/tests
```

`test-all` is `test-native-cmake` → `test-cortex-cmake` → `test-smp-cmake`: every host compiler (GCC 11–14 and clang 16–19 on Linux; the system compiler and clang 16–19 on macOS — clang 13–15 are listed but disabled), the four QEMU Cortex-M cores, then every SMP platform (AArch32, AArch64, the emulated Cortex-M33 pair, the Pico 2 boards under QEMU), each in debug and release. Hardware (`hwd`) cases are **never** part of it. Expect it to take hours.

Remove all dependencies and build folders, then restart from `npm install`:

```
xpm run deep-clean -C ~/Work/micro-os-plus/micro-os-plus-iii/tests
```

Never `rm -rf tests/build/<C>` **by hand to "fix" a build.** The folder also holds that configuration's pinned toolchain (`xpacks/`). If a CMake cache is poisoned, delete only `tests/build/<C>/CMakeCache.txt` and run `prepare` again.

4. Architecture in one picture

Four layers, each edited by hand, each with one job:

```
xpm run test-foo-qemu --config cortexm-pico2-cmake-gcc-debug
|
| LAYER 1  tests/package.json
|          which configuration, which toolchain, which build type,
|          which actions exist (and therefore which appear in VS Code)
|
▼
cmake -S tests -B build/<C> -G Ninja -D PLATFORM_NAME=cortexm-pico2
      -D CMAKE_BUILD_TYPE=Debug -D CMAKE_TOOLCHAIN_FILE=.../arm-none-eabi-gcc.cmake
|
| LAYER 2  tests/CMakeLists.txt → cmake/tests-main.cmake
|          → tests/platforms/<platform>/
|          which port (library under test) and which CTest cases exist
|
▼
add_subdirectory(<port>/test)          ← the port's own test builder
|
| LAYER 3  <port>/test/CMakeLists.txt + test/boards/<board>/board.cmake
|          + test/<board>/tests.cmake + test/<board>/<app>/*.cpp
|          which images are built: <app>-qemu, <app>-hwd, <app>-host
|
▼
ctest -R cortexm-pico2-foo-qemu
|
| LAYER 4  CTest case → runner (qemu / run-qemu.sh / run-host.sh / hw.sh)
|          the verdict: exit code and the "RESULT: PASS|FAIL|SKIP" line
|
▼
PASS / FAIL / TIMEOUT
```

The rule that makes this work: **only** `package.json` **passes** `-D PLATFORM_NAME`, `CMAKE_BUILD_TYPE` and `CMAKE_TOOLCHAIN_FILE`, and only the platform folder creates CTest cases. A port never knows it is inside the harness; a platform never compiles a test by itself when the port can.

5. Layer 1 — `tests/package.json` (xpm)

The file has two parts that matter: `xpack.properties` + `xpack.actions` (global), and `xpack.buildConfigurations` (one entry per configuration, plus hidden mixins).

5.1 Properties: the command chain

Properties are [Liquid](#) templates, expanded by xpm before an action runs. The global ones build every CMake command:

property	expands to
<code>buildFolderRelativePath</code>	<code>build/<configuration name, lower-cased></code>
<code>commandCMakeReconfigure</code>	<code>cmake -S . -B <build> -G Ninja -D CMAKE_BUILD_TYPE={{buildType}} -D PLATFORM_NAME={{platformName}}</code>
<code>commandCMakePrepare</code>	<code><Reconfigure> --log-level=VERBOSE</code>
<code>commandCMakePrepareWithToolchain</code>	<code><Prepare> -D CMAKE_TOOLCHAIN_FILE=xpacks/@micro-os-plus/build-helper/cmake/toolchains/{{toolchainFileName}}</code>
<code>commandCMakeBuild</code>	<code>cmake --build <build></code>
<code>commandCMakePerformTests</code>	<code>cd <build> && ctest -V</code>

A configuration supplies the variables they use: `buildType`, `platformName`, `toolchainFileName`, and (for short Windows paths) `shortConfigurationName`.

5.2 Hidden mixins

A configuration with `"hidden": true` has no build folder; it exists to be inherited. The ones in use:

mixin	provides
<code>cmake-actions</code>	the lifecycle actions <code>prepare</code> , <code>build</code> , <code>test</code> , <code>clean</code>
<code>native-actions</code> , <code>cortexm-actions</code> , <code>aarch32-actions</code> , <code>aarch64-actions</code>	<code>install (xpm install --config {{ configuration.name }})</code> , plus <code>link-deps</code> for native/cortexm
<code>gccNN-dependencies</code> , <code>clangNN-dependencies</code> , <code>gcc-latest-...</code> , <code>clang-latest-...</code>	the pinned host compiler
<code>arm-none-eabi-gcc-dependencies</code> , <code>aarch32-dependencies</code> , <code>aarch64-dependencies</code>	the cross toolchains (+ <code>qemu-arm</code> for the Pis)
<code>qemu-arm-dependencies</code> , <code>openocd-dependencies</code>	emulator / probe tools
<code>native-dependencies</code> , <code>cortexm-dependencies</code> , <code>arm-cmsis-...</code>	source packages the platform needs
<code>clang-gcc14-properties</code>	for clang 16–18: builds against the xPack GCC 14 libstdc++ (<code>--gcc-toolchain=...</code>), because clang < 19 cannot parse a newer host GCC's headers
<code>short-win-paths-properties</code>	on Windows, <code>build/<shortConfigurationName></code> instead of the long name

5.3 A concrete configuration

Name pattern: `<platform>-cmake-<toolchain>-<debug|release>`.

```

"cortexm-pico2-rp2350b-psram-cmake-gcc-debug": {
  "inherit": [
    "cortexm-actions", "cmake-actions", "cortexm-dependencies",
    "arm-cmsis-dependencies", "arm-none-eabi-gcc-dependencies",
    "openocd-dependencies", "short-win-paths-properties"
  ],
  "properties": {
    "buildType": "Debug",
    "platformName": "cortexm-pico2-rp2350b-psram",
    "toolchainFileName": "arm-none-eabi-gcc.cmake",
    "shortConfigurationName": "cpsd"
  },
  "actions": {
    "test": "cd {{ properties.buildFolderRelativePath }} && ctest -V -LE hwd",
    "test-fp-switch-qemu": "{{ properties.commandCMakePrepareWithToolchain }} && {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }} && ctest -V -R cortexm-pico2-rp2350b-psram-fp-switch-qemu",
    "test-fp-switch-hwd": "... same, ending in -fp-switch-hwd"
  }
},
"cortexm-pico2-rp2350b-psram-cmake-gcc-release": {
  "inherit": ["cortexm-pico2-rp2350b-psram-cmake-gcc-debug"],
  "properties": { "buildType": "MinSizeRel", "shortConfigurationName": "cpsr" }
}

```

Inheritance rules (xpm):

- parents are merged in the order listed; a later parent wins over an earlier one; the configuration's own `properties` / `actions` win over all;
- a local action **replaces** an inherited action of the same name (that is how `test` above becomes `ctest -V -LE hwd`);
- the **release** configuration inherits the debug one and changes only `buildType` (`MinSizeRel` on embedded, `Release` on native). It has its **own build folder**, so it needs its own `install`.

5.4 Actions: three kinds

Lifecycle (per configuration, from `cmake-actions` and the family mixin):

action	does	note
<code>install</code>	downloads the configuration's pinned tools into <code>build/<C>/xpacks</code>	run once per configuration
<code>prepare</code>	CMake configure with the toolchain file	must precede <code>build</code> on a new folder
<code>build</code>	CMake reconfigure (no toolchain arg — it is already cached) + build	
<code>test</code>	<code>ctest -V</code> — or <code>ctest -V -LE hwd</code> on every configuration that has hardware cases	does not build
<code>clean</code>	<code>cmake --build ... --target clean</code>	

Per-test (per configuration, written by hand, one per CTest case):

```

"test-<app>-<variant>": "{{ properties.commandCMakePrepareWithToolchain }} && {{ properties.commandCMakeBuild }} && cd {{ properties.buildFolderRelativePath }} && ctest -V -R <platformName>-<app>-<variant>"

```

It configures and builds before it runs, so it works on a cold tree once `install` has run. **These actions are what make a test appear in the VS Code xPack extension**; there is no generator, so a test without its action runs in CTest but is invisible in the IDE.

For the upstream-style platforms (`qemu-cortex-m*`, `pico2-1cpu`, `2xcortex-m33`) the case names are `<platform>-<suite>-test`, so the actions are `test-<suite>-test`.

Top-level (`xpack.actions`, no `--config`), all arrays run in sequence and stopping at the first failure:

action	runs
<code>install</code> , <code>test</code>	the quick set (<code>qemu-cortex-m7f</code> , debug + release)
<code>install-all</code>	<code>npm install</code> + <code>xpm install --all-configs</code>
<code>test-all</code>	<code>test-native-cmake</code> + <code>test-cortex-cmake</code> + <code>test-smp-cmake</code>
<code>test-native-cmake</code>	every <code>test-native-cmake-<compiler></code> : Linux GCC 11–14 + clang 16–19, macOS <code>sys</code> + clang 16–19 (clang 13–15 are only echoed, i.e. disabled)
<code>test-native-cmake-<compiler></code>	prepare/build/test of that compiler's debug and release
<code>test-cortex-cmake</code>	the four <code>test-qemu-cortex-m*-cmake</code>
<code>test-smp-cmake</code>	every SMP platform's <code>test-<platform>-cmake</code>
<code>test-<platform>-cmake</code>	prepare/build/test, debug then release, of one platform
<code>build-aarch32-luckfox-lyra-cmake</code>	build only (the Lyra is hardware-only)
<code>clean-all</code> , <code>clean-native-cmake</code> , <code>clean-cortex-cmake</code>	<code>clean</code> across configurations
<code>link-deps</code> , <code>link-deps-all</code> , <code>git-clone-deps</code> , <code>git-pull-deps</code> , <code>git-status-deps</code>	writable development packages (upstream workflow)
<code>deep-clean</code>	removes <code>build/</code> , <code>node_modules/</code> , <code>xpacks/</code> , <code>package-lock.json</code>

An array action accepts no extra command-line arguments.

6. Layer 2 — the CMake harness (`tests/`)

```

tests/
├─ CMakeLists.txt      project, C11/C++20, enable_testing(), build helper,
│                      then cmake/global-definitions.cmake + cmake/tests-main.cmake
├─ cmake/
│   ├─ tests-main.cmake  chooses the port from PLATFORM_NAME, adds the platform
│   ├─ common-options.cmake warnings and options shared by every target
│   └─ global-definitions.cmake ENABLE_*_TEST switches (read by upstream platforms only)
├─ platforms/<platform>/  one folder per PLATFORM_NAME (§6.4)
├─ sources/<suite>/        the harness suites (§10.3)
├─ device-qemu-cortexm/    device support for the QEMU Cortex-M boards
└─ package.json           Layer 1

```

6.1 tests/CMakeLists.txt

Refuses an in-source build, sets the language standards, loads `micro-os-plus-build-helper` from `xpacks/@micro-os-plus/build-helper`, and includes `global-definitions.cmake` and `tests-main.cmake`. You should not need to edit it.

6.2 cmake/tests-main.cmake — which library is under test

In order:

1. includes `platforms/${PLATFORM_NAME}/cmake/definitions.cmake` and `../dependencies-folders.cmake`, and adds each folder listed in `xpack_dependencies_folders` (the suites, CMSIS, ...);
2. resolves the sibling repositories into cache variables (§6.3);
3. adds **the library under test**, chosen by the **first matching prefix** of `PLATFORM_NAME`:

<code>PLATFORM_NAME</code> starts with	added as <code>port-bin</code>	platforms
<code>aarch32</code>	<code>UOS_AARCH32_DIR</code>	<code>aarch32-rpi-zero-2w</code> , <code>aarch32-rpi3b</code> , <code>aarch32-luckfox-lyra</code>
<code>aarch64</code>	<code>UOS_AARCH64_DIR</code>	<code>aarch64-rpi-zero-2w</code> , <code>aarch64-rpi3b</code>
<code>native</code>	<code>UOS_POSIX_ARCH_DIR</code>	<code>native</code>
<code>cortexm</code> , <code>qemu-cortex</code>	<code>UOS_CORTEXM_DIR</code>	<code>cortexm-*</code> boards, <code>qemu-cortex-m0/m3/m4f/m7f</code>
<code>pico2</code> , <code>2xcortex</code>	<code>UOS_CORTEXM_DIR</code>	<code>pico2-1cpu</code> , <code>2xcortex-m33</code>
anything else	the kernel itself (<code>..</code> , as <code>top-bin</code>)	<code>nucleo-*</code> , <code>raspberrypi-pico</code>

Each port brings the kernel (`micro-os-plus::iii`) and the devices library with it; **never add the kernel a second time** (`alias micro-os-plus::iii` already exists).

1. finally `add_subdirectory(platforms/${PLATFORM_NAME} platform-bin)`.

6.3 Where the repositories come from

cache variable	default
<code>UOS_SMP_DIR</code>	<code>tests/..</code> (this kernel, forced)
<code>UOS_AARCH32_DIR</code>	<code><tests>/../../micro-os-plus-iii-aarch32</code>
<code>UOS_AARCH64_DIR</code>	<code><tests>/../../micro-os-plus-iii-aarch64</code>
<code>UOS_POSIX_ARCH_DIR</code>	<code><tests>/../../micro-os-plus-iii-posix-arch</code>
<code>UOS_CORTEXM_DIR</code>	<code><tests>/../../micro-os-plus-iii-cortexm</code>

If the plain name has no `CMakeLists.txt` but `<name>.git` does, the `.git` name is used. Override on the command line to test another working copy:

```
cd ~/Work/micro-os-plus/micro-os-plus-iii/tests
PATH="$PWD/build/C/xpacks/.bin:$PATH" \
  cmake -S . -B build/C -D UOS_CORTEXM_DIR=$HOME/src/my-cortexm
```

6.4 A platform folder

```
tests/platforms/<platform>/
├── CMakeLists.txt           registers the CTest cases           ($6.5)
├── cmake/
│   ├── definitions.cmake    xpack_platform_compile_definition;
│   │                       on a port platform: set (BOARD "<id>" CACHE STRING ""
│   │                       FORCE)
│   ├── dependencies-folders.cmake xpack_dependencies_folders: sources/<suite>, CMSIS, ...
│   └── platform-library.cmake micro-os-plus::platform (flags, linker, port libs)
│                               [+ micro-os-plus::platform-support on some platforms]
├── include/cmsis-plus/platform.h compile-time contract, included by each suite's config
└── src/platform-support.cpp startup hooks + a strong main() calling os_main()
                               (only platforms that run harness suites through it)
```

`dependencies-folders.cmake` is per platform. A suite that is not listed there does not exist on that platform: `test::<suite>` is undefined and the configure fails.

6.5 The two shapes of a platform `CMakeLists.txt`

(a) Port platforms (`aarch32-*`, `aarch64-*`, `cortexm-*`, `native`) reuse the port's own builder and **glob** the board's test folders:

```
include ("cmake/platform-library.cmake")
include ("${UOS_CORTEXM_DIR}/test/boards/${BOARD}/board.cmake") # board facts
add_subdirectory ("${UOS_CORTEXM_DIR}/test" "port-tests/test") # builds the images

file (GLOB _test_dirs LIST_DIRECTORIES true "${UOS_CORTEXM_DIR}/test/${BOARD}/*")
foreach (_dir IN LISTS _test_dirs)
    # skip files, include/, src/ and folders without *.c/*.cpp
    if (TARGET "${_app}-qemu")
        add_test (NAME "${PLATFORM_NAME}-${_app}-qemu" COMMAND <qemu> ... -kernel .../${_app}-qemu.elf
            ...)
        set_tests_properties (... PROPERTIES LABELS "qemu" TIMEOUT 300)
    endif ()
    add_test (NAME "${PLATFORM_NAME}-${_app}-hwd" COMMAND bash <board>/hw.sh "${_app}" 600)
    set_tests_properties (... PROPERTIES ENVIRONMENT "BUILD=..." LABELS "hwd" TIMEOUT 1200)
endforeach ()
```

Consequence: **adding a board test needs no edit in** `tests/platforms/`. Creating the folder in the port and re-running `prepare` is enough for the CTest case to exist. (The xpm action is still yours to add — §5.4.)

The differences between the port platforms:

platform family	-qemu case runs	-host	-hwd case runs
<code>aarch32-rpi*</code> , <code>aarch64-rpi*</code>	<code>test_smpl/run-qemu.sh <bin> <qemu> -M raspi3b -smp 4 (UOS_QEMU_ONLY=<app>)</code>	—	the board's <code>hw.sh</code> <code><app> 600</code>

platform family	-qemu case runs	-host	-hwd case runs
aarch32-luckfox-lyra	— (hardware only)	—	the board's hw.sh
cortexm-*	qemu-system-arm -M mps2-an500 -cpu cortex-m7 -kernel <app>-qemu.elf ..., only if the port built <app>-qemu	—	the board's hw.sh <app> 600
native	—	test_smpl/run-host.sh <bin> (UOS_RUN_ONLY=<app>)	—

(b) Upstream-style platforms (qemu-cortex-m*, pico2-1cpu, 2xcortex-m33, nucleo-*, raspberrypi-pico) build the harness suites themselves:

```
function (add_suite_executable name suite)
  add_executable (${name})
  target_link_libraries (${name} PRIVATE micro-os-plus::common-options
                                test::${suite} micro-os-plus::iii micro-os-plus::platform ${ARGN})
  add_test (NAME "${PLATFORM_NAME}-${name}"
            COMMAND "${_qemu}" --machine mps2-an500 --cpu cortex-m7
                    --kernel "${TARGET_FILE:${name}}>" --nographic
                    --semihosting-config enable=on,target=native)
  set_tests_properties ("${PLATFORM_NAME}-${name}" PROPERTIES LABELS "qemu" TIMEOUT 1200)
endfunction ()

add_suite_executable (rtos-apis-test          rtos-apis          xpacks::chan-fatfs)
add_suite_executable (mutex-stress-test       mutex-stress)
add_suite_executable (cmsis-os-validator-test cmsis-os-validator xpacks::arm-cmsis-os-validator)
```

The QEMU binary is found by globbing `$HOME/.local/xPacks/@xpack-dev-tools/qemu-arm/*/content/bin` (newest first), falling back to `PATH`.

7. Layer 3 — the port test builders

Every port carries the same structure; the harness only includes it.

```
<port>/test/
├─ CMakeLists.txt           the builder: one application per folder of test/<BOARD>/
├─ boards/<board>/
│   └─ board.cmake          board facts: UOS_BOARD_NCPU, UOS_BOARD_LINKER_HW,
│                           UOS_BOARD_LINKER_QEMU, UOS_BOARD_DEFINES, sources, ...
│                           hardware runner for this board (OpenOCD)
│                           probe + target
├─ hw.sh                    cross-board helpers, e.g. hw_result.hpp (cortexm)
├─ openocd.cfg              the board's TESTS – one folder per application
├─ boards/shared/           support shared by this board's tests (not applications)
└─ <board>/
    ├─ include/ src/         the per-test knobs a folder listing cannot express
    ├─ tests.cmake           one test
    └─ <app>/main.cpp ...
```

7.1 What the builder produces

For each folder `test/<BOARD>/<app>/` that contains a `*.cpp` — `*.c` alone does not count (skipping `include/` and `src/`):

port	images
<code>cortexm</code>	<code><app>-qemu.elf</code> if <code>board.cmake</code> sets <code>UOS_BOARD_LINKER_QEMU</code> and <code><app></code> is not in <code>BOARD_TEST_HWD_ONLY</code> ; <code><app>-hwd</code> always
<code>aarch32</code> , <code>aarch64</code>	<code><app>-qemu.bin</code> and <code><app>-hwd</code>
<code>posix-arch</code>	<code><app>-host</code>

Tests are never shared between boards. Two boards of the same SoC each have their own `test/<board>/`; changing a test reaches one board only.

7.2 `tests.cmake` — the per-test knobs

Lists (variables) and hooks (functions `board_test_<x> (_app _out)` that set `${_out}` in `PARENT_SCOPE`). The builder defines empty defaults; the board's `tests.cmake` overrides what it needs.

knob	kind	ports	effect
<code>BOARD_TEST_NEED_DEVICES</code>	list	all	link the board's device drivers (<code>UOS_BOARD_DEVICES</code>)
<code>BOARD_TEST_SELF_CONTAINED</code>	list	all	the test brings its own startup / board glue
<code>BOARD_TEST_NO_KERNEL</code>	list	all	bare-metal: no kernel, no scheduler, no <code>OS_NCPU</code>
<code>BOARD_TEST_HWD_ONLY</code>	list	<code>cortexm</code>	build <code>-hwd</code> only (QEMU cannot model what it tests)
<code>board_test_ncpu</code>	hook	all	<code>OS_NCPU</code> for this app (e.g. 1 for single-core tests)
<code>board_test_defines</code>	hook	all	extra compile definitions
<code>board_test_sources</code>	hook	all	extra sources (board glue, PSRAM, ADC, ...)
<code>board_test_includes</code>	hook	all	extra include directories
<code>board_test_linker</code>	hook	all	a different linker script (e.g. PSRAM)
<code>board_test_options</code>	hook	all	extra compile options (e.g. <code>-mlong-calls</code>)
<code>board_test_libs</code>	hook	<code>cortexm</code> , <code>aarch32/64</code>	extra link libraries — used to turn a harness suite into a board app (§10.4)
<code>board_test_libraries</code>	hook	<code>posix-arch</code>	same, <code>posix-arch</code> spelling
<code>board_test_qemu_libs/_sources/_includes</code>	hooks	<code>cortexm</code>	replace the board for the <code>-qemu</code> image (the generic single-core QEMU core)

A test that needs none of these needs **no** `tests.cmake` edit.

8. Layer 4 — CTest, runners and verdicts

8.1 Case names and labels

platform shape	CTest name	label
port platform	<code><platform>-<app>-qemu / -hwd / -host</code>	<code>qemu / hwd / host</code>
upstream-style platform	<code><platform>-<suite>-test</code>	<code>qemu (hwd on the nucleo / pico boards)</code>

`ctest -R` is a **regular-expression substring** match: always pass the full name, or `-R cortexm-pico2-smp-test1-qemu` also runs `cortexm-pico2-smp-test1-qemu-something`.

The generic `test` action runs `ctest -V -LE hwd`: **hardware cases are excluded on purpose**, because each one needs a board and a power cycle.

8.2 The verdict protocol

A test passes by saying so, in one of two ways:

mechanism	used by	how
exit status	upstream-style suites; cortexm <code>-qemu</code> ; native <code>cmsis-os-validator</code>	semihosting <code>SYS_EXIT</code> (QEMU returns it as the process status) or the host process exit code; <code>0</code> = pass
RESULT: line	runners (<code>run-qemu.sh</code> , <code>run-host.sh</code> , <code>run-hw.sh</code> , the boards' <code>hw.sh</code>)	the log is grepped for <code>RESULT: PASS</code> , <code>RESULT: SKIP</code> , <code>RESULT: FAIL</code>

Board tests do both: print `RESULT: PASS|FAIL` and then **stop the run** — `hw_result::ok()` / `hw_result::fail()` (cortexm `boards/shared/hw_result.hpp`, and the aarch/posix equivalents) write the line through semihosting and call `std::_Exit(0|1)`. A test that prints `RESULT:` but never stops is reported as a timeout on hardware.

8.3 The runners (`test_smp1/`)

script	arguments	environment	logs
<code>run-qemu.sh</code>	<code><build-test-dir></code> <code><qemu> <machine-args...></code>	<code>UOS_QEMU_ONLY=<app></code> , <code>UOS_QEMU_SHIM /</code> <code>UOS_QEMU_LOAD_ADDR</code> (AArch32 on raspi3b), <code>UOS_TEST_SRC_DIR</code> (SD image seeding)	<code><dir>/ .qemu-logs/</code> <code><app>.log</code>
<code>run-host.sh</code>	<code><build-test-dir></code>	<code>UOS_RUN_ONLY=<app></code>	<code><dir>/ .host-logs/</code> <code><app>.log</code>
<code>run-hw.sh</code>	<code><build-test-dir></code> <code><app> [secs]</code> or <code>list</code>	<code>UOS_HW_CFG</code> (required), <code>UOS_HW_*</code>	<code><dir>/ .hw-logs/</code> <code><app>.log</code>

Verdicts: `PASS`, `SKIP` (e.g. `usb_test` under QEMU), `FAIL`, `TIMEOUT`, `NO RESULT`. Per-test time budgets are in `timeout_for()` of `run-qemu.sh` and `run-host.sh`: 300 s unless listed (`smp_test0/1/3/4` 150, `smp_test2` 300, `smp-mat-test` 900, `smp-num/pipeline/pro-cons` 1000, `smp-mat-sdcard-test` 2000, `sd_test` 450, `usb_test` 200). SD-card tests get a fresh 4 GiB sparse card image per run, deleted afterwards.

8.4 Hardware runs (`-hwd`)

- **OpenOCD only, no GDB.** The runner programs the image, enables semihosting and resumes; the verdict comes from the semihosting output.
- **The UART is yours.** No runner opens the serial port; keep your terminal (`tio` , `picocom`) on it.
- **One test per power cycle.** Power-cycle the board before each `-hwd` case; `run-hw.sh` refuses to run a suite.

9. Running tests day to day

All commands run in `~/Work/micro-os-plus/micro-os-plus-iii/tests` (or add `-C ~/Work/micro-os-plus/micro-os-plus-iii/tests`). `C` is a configuration name.

goal	command
tools, first time	<code>npm install && xpm install</code>
one configuration, cold	<code>xpm run install --config C</code> → <code>xpm run prepare --config C</code> → <code>xpm run build --config C</code> → <code>xpm run test --config C</code> (this order)
one platform, debug + release	<code>xpm run test-<platform>-cmake</code>
one compiler, debug + release	<code>xpm run test-native-cmake-gcc14</code>
one test (builds first)	<code>xpm run test-<app>-<variant> --config C</code>
list the cases	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" ctest --test-dir build/C -N</code>
one case, by hand	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" ctest --test-dir build/C -V -R '^<platform>-<app>-<variant>\$'</code>
rebuild after editing a test	<code>xpm run build --config C</code> (or the per-test action)
a new test folder was added	<code>xpm run prepare --config C</code> — the glob runs at configure time
every broken target at once	<code>PATH="\$PWD/build/C/xpacks/.bin:\$PATH" cmake --build build/C -- -k 0</code>
the log of the last run	<code>build/C/.../.qemu-logs/<app>.log</code> , <code>.host-logs/</code> , <code>.hw-logs/</code>

Standing rules:

- Run QEMU suites **one at a time** on an idle host; parallel emulators turn timing-sensitive SMP tests into timeouts. Re-run a lone timeout by itself before debugging it.
- `test` never builds. After an edit, `build` first or use the per-test action.
- A release configuration is separate: `install` it before its first `prepare` .

10. Writing a new test

10.1 Which kind of test?

the test...	it is a...	lives in	its verdict
is portable, has no <code>main()</code> , entry <code>os_main()</code>	harness suite	<code>tests/sources/<name>/</code>	<code>os_main()</code> return value → exit code
needs one board, has its own <code>main()</code> / hooks	board test	<code><port>/test/<board>/<name>/</code>	prints <code>RESULT:</code> and stops

Rule of thumb: a test of **kernel API behaviour** that should hold on every architecture is a harness suite. A test of a **board, a driver, an SMP boot sequence or a port detail** is a board test.

A suite links `micro-os-plus::platform` and the platform's startup support (`platform-support`, or the board's harness wrapper); a board test links the platform **only**, because it brings its own `main()` — two strong `main()`s collide at link time.

10.2 Recipe A — a board test

1. **Create the folder** `<port>/test/<board>/<name>/` with a `main.cpp`. Start from the closest sibling in the same folder (e.g. `smp-test1/` on a Pico 2 board, `smp_test0/` on native). Keep its boot sequence; change the body. The folder name **is** the application name.
2. **End with a verdict** — print the line, then stop:

```
#include <hw_result.hpp>
// ...
if (ok)
{
    uart::write ("RESULT: PASS\n"); // the board's console, for you
    hw_result::ok ();               // semihosting line + _Exit(0): stops the run
}
else
{
    uart::write ("RESULT: FAIL\n");
    hw_result::fail ();             // _Exit(1)
}
for (;;) {}                       // reached only without semihosting
```

3. **If needed, declare it in** `<port>/test/<board>/tests.cmake` (§7.2): `BOARD_TEST_NEED_DEVICES`, `BOARD_TEST_NO_KERNEL`, `BOARD_TEST_HWD_ONLY` (cortexm), a different `board_test_ncpu`, extra sources/defines. Most tests need nothing.
4. **If it runs longer than 300 s** under QEMU or on the host, add it to `timeout_for()` in `test_smp1/run-qemu.sh` / `run-host.sh`.
5. **Re-configure** so the glob sees it, and check the case exists:

```
xpm run prepare --config C
PATH="$PWD/build/C/xpacks/.bin:$PATH" ctest --test-dir build/C -N | grep <name>
```

6. Add one action per variant to each configuration that builds it (the debug one; release inherits it):

```
"test-<name>-qemu": "{ { properties.commandCMakePrepareWithToolchain } } &&
    { { properties.commandCMakeBuild } } && cd { { properties.buildFolderRelativePath } }
    && ctest -V -R <platform>-<name>-qemu",
"test-<name>-hwd": "{ { properties.commandCMakePrepareWithToolchain } } &&
    { { properties.commandCMakeBuild } } && cd { { properties.buildFolderRelativePath } }
    && ctest -V -R <platform>-<name>-hwd"
```

7. Build everything and run it:

```
PATH="$PWD/build/C/xpacks/.bin:$PATH" cmake --build build/C -- -k 0
xpm run test-<name>-qemu --config C
```

A per-test action builds the **whole** tree: one target that does not build makes every action of that configuration fail.

8. **Hardware**: power-cycle the board, then `xpm run test-<name>-hwd --config C`.

10.3 Recipe B — a harness suite

1. Create the files, copying `tests/sources/mutex-stress/`:

```
tests/sources/<name>/
├─ CMakeLists.txt
├─ README.md
├─ include/cmsis-plus/os-app-config.h    per-suite RTOS configuration
└─ src/main.cpp                        int os_main (int argc, char* argv[])
```

2. `CMakeLists.txt` — an INTERFACE library with the `test::` alias:

```
cmake_minimum_required (VERSION 3.20)

add_library (test-<name>-interface INTERFACE EXCLUDE_FROM_ALL)
target_include_directories (test-<name>-interface INTERFACE "include")
target_sources (test-<name>-interface INTERFACE src/main.cpp)

if (COMMAND xpack_display_target_lists)
    xpack_display_target_lists (test-<name>-interface)
endif ()

add_library (test::<name> ALIAS test-<name>-interface)
message (VERBOSE "> test::<name> -> test-<name>-interface")
```

3. `src/main.cpp` — the entry point is `os_main`, never `main`; return 0 for pass:

```

#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/diag/trace.h>

using namespace os;
using namespace os::rtos;

int
os_main (int argc __attribute__ ((unused)),
         char* argv[] __attribute__ ((unused)))
{
    bool ok = true;
    // ... exercise the kernel API; set ok = false on any failure ...
    return ok ? 0 : 1;
}

```

Do not trace from the scheduler/tick ISR. Keep the controlling thread above the priority of busy worker threads, or on real silicon it may never run again (this is what `fp-switch` does).

4. `include/cmsis-plus/os-app-config.h` — copy the one from `mutex-stress` (or `fp-switch`); it includes `cmsis-plus/platform.h` and sets the tick, stack sizes and per-architecture options.

5. Wire it into each platform that should run it:

- add `"${CMAKE_SOURCE_DIR}/sources/<name>"` to that platform's `cmake/dependencies-folders.cmake`;
- upstream-style platform: add to its `CMakeLists.txt` `add_suite_executable (<name>-test <name> [extra libs])`;
- port platform (cortexm, aarch32/64): run it as a board app — §10.4.

6. `prepare`, add the `test-<name>-test` (or `test-<name>-<variant>`) action, full build with `-k 0`, run.

10.4 Recipe C — a harness suite as a board app (port platforms)

Port platforms build only what the port builder builds, so a suite runs there as a **board app** that wraps `os_main()`. This is how `cortexm-pico2` runs `rtos-apis`, `mutex-stress`, `cmsis-os-validator` and `fp-switch`, and how `cortexm-pico2-rp2350b-psram` runs `fp-switch`. Every step is required:

1. **A folder per suite:** `<port>/test/<board>/<suite>/harness-suite.cpp` (the folder gives the app its name). On the Pico 2 boards it is one line:

```
#include "../pico2/harness-suite.hpp"
```

`harness-suite.hpp` defines `__wrap_os_main()`: it calls `__real_os_main()`, prints `RESULT: PASS|FAIL` on the UART **and** through semihosting, and returns the code; the kernel's `std::exit(code)` is the semihosting `SYS_EXIT`.

2. `board_test_libs()` in the board's `tests.cmake` returns `test::<suite>` (+ its extras, e.g. `xpacks::chan-fatfs`) **and** an INTERFACE target that carries:
 - `OS_USE_OS_APP_CONFIG_H UOS_HARNESS_SUITE _POSIX_C_SOURCE=200809L _GNU_SOURCE`;
 - the include directory `${CMAKE_SOURCE_DIR}/platforms/${PLATFORM_NAME}/include` (the harness `platform.h` the suite's config includes);
 - `micro-os-plus::iii-semihosting`, `iii-newlib-reent`, `iii-posix-io`;

- the link option `-Wl,--wrap=os_main`.

On cortexm this target is `pico2-harness-suite`; on the Pis it is `<platform>-harness-suite` and also adds the platform's `src/platform-support.cpp`.

3. `board_test_ncpu()` returns 1 for the suite if the suite is single-core.
4. The suite folder must be in the platform's `dependencies-folders.cmake` (Recipe B, step 5), or `test::<suite>` is undefined.
5. `prepare`, add the `-qemu` / `-hwd` actions, full build, run.

The board's own `os-app-config.h` must `#include_next` the suite's config under `UOS_HARNESS_SUITE` and wrap its own values in `#ifndef`, so the suite's settings win and the board's other tests are unchanged.

11. Adding a configuration or a platform

11.1 A new toolchain or build type for an existing platform

1. Copy the nearest **pair** (debug + release) in `xpack.buildConfigurations`, rename it `<platform>-cmake-<toolchain>-<debug|release>`.
2. Change the dependency mixin (`gccNN-dependencies`, ...), the `toolchainFileName`, and a unique `shortConfigurationName`.
3. Copy the `test` override and the per-test actions from the sibling.
4. Add it to the relevant top-level array (`test-native-cmake`, ...) and give it a `test-<platform>-cmake` -style action if it needs one.
5. `xpm install --config <new>-debug`, then the cold sequence of §9.

For clang < 19 on a host with a newer GCC, inherit `gcc14-dependencies` and `clang-gcc14-properties` as `native-cmake-clang16/17/18-*` do.

11.2 A new platform

1. **Name it** with a prefix that `tests-main.cmake` maps to the right port (§6.2), or add a branch there.
2. **Copy the nearest sibling folder** under `tests/platforms/` and adapt:
 - `cmake/definitions.cmake`: `xpack_platform_compile_definition`, and `set (BOARD "<board>" CACHE STRING "" FORCE)` for a port platform;
 - `cmake/dependencies-folders.cmake`: the suites and packages it needs;
 - `cmake/platform-library.cmake`: `micro-os-plus::platform` (the port library, startup, semihosting, flags);
 - `CMakeLists.txt`: the shape of §6.5 (a) or (b);
 - `include/cmsis-plus/platform.h`.
3. **On the port side** (port platforms): `test/boards/<board>/board.cmake`, `hw.sh`, `openocd.cfg`, and `test/<board>/` with its tests and `tests.cmake`. The builder fails at configure time if `test/<board>/` is missing.
4. **Add the configuration pair** (§11.1) with the right dependency mixins: an emulator needs `qemu-arm-dependencies`, a board needs `openocd-dependencies`.
5. Add `test-<platform>-cmake` and, if it is emulated, list it in `test-smp-cmake` (or the relevant top-level set).
6. Re-run the emulated sets of the **other** platforms of the same port: a new board must not change their results.

Some files carry a "DO NOT EDIT ... generated from build-helper" header (`cmake/tests-main.cmake` , `cmake/common-options.cmake` , several upstream `platform-library.cmake`). This tree has diverged from those templates on purpose: edit the local copy, do not regenerate it.

12. Checklist before you commit

☐

`xpm run prepare --config C` done after adding or removing a test folder.

☐

`ctest -N` shows the new case with the full name you expect.

☐

`cmake --build build/C -- -k 0` builds **every** target (not only yours).

☐

Every CTest case of every configuration that builds it has a `test-<app>-<variant>` action. A quick cross-check:

```
```sh
cd ~/Work/micro-os-plus/micro-os-plus-iii/tests
C=cortexm-pico2-cmake-gcc-debug
PATH="$PWD/build/$C/xpacks/.bin:$PATH" ctest --test-dir build/$C -N \
 | sed -n 's/.*Test *#[0-9]*: //p' | while read -r t; do
 grep -q "ctest -V -R $t\" package.json || echo "missing action: $t"
 done
```
```

☐

The test passes alone, and in its configuration's full `test` run.

☐

Debug **and** release (release often exposes timing and optimisation issues — see the clang TLS defect in `docs/posix-arch-port.md` §2).

☐

For a port change: the emulated sets of the port's other boards still pass.

☐

For hardware: the `-hwd` case passes on the board, one per power cycle.

☐

`docs/tests/TESTS-CATALOG.md` updated if you added a test or a platform.

13. Troubleshooting

| symptom | cause and fix |
|---|---|
| <code>found /usr/bin/cc</code> , or the toolchain guard says the compiler "must be ..." | configured before <code>install</code> , or <code>build</code> before <code>prepare</code> . Run <code>install</code> → <code>prepare</code> again. If the cache is poisoned, delete only <code>build/C/CMakeCache.txt</code> . |
| Missing <code>.../build/C/xpacks/...</code> on a release configuration | release has its own build folder: <code>xpm run install --config <release></code> |
| no <code>*-host executables</code> / every case fails at once | <code>test</code> does not build: run <code>build</code> first, or use the per-test action |
| the test runs in CTest but not in VS Code | its <code>test-<app>-<variant></code> action is missing from <code>package.json</code> |
| every <code>test-*</code> action of one configuration fails at the build step | one target in that tree does not build: <code>cmake --build build/C -- -k 0</code> lists them all |
| a new port test is not in <code>ctest -N</code> | not re-prepared; or the folder has no <code>*.cpp</code> (the builders glob <code>*.cpp</code> only — a folder with just <code>*.c</code> gets a CTest case on cortexm but no image); or the port clone is not the one CMake uses (<code>UOS_*_DIR</code> in <code>CMakeCache.txt</code>) |
| Cannot find the <code><port></code> port at ... | the sibling repository is missing or misnamed (§2); pass <code>-D UOS_<PORT>_DIR=...</code> |
| <code>alias micro-os-plus::iii already exists</code> | the kernel was added twice; the port adds it — never add it again |
| undefined reference to <code>os_startup_initialize_hardware...</code> | a harness suite linked without the platform support / harness wrapper |
| undefined reference to <code>test::<suite></code> / target not found | the suite is not in the platform's <code>dependencies-folders.cmake</code> |
| two definitions of <code>main</code> | a board test linked with <code>platform-support</code> , or a suite given its own <code>main()</code> |
| a QEMU case times out with no fault in the log | host contention (parallel emulators) or a missing <code>timeout_for</code> entry: re-run alone |
| <code>sd_test</code> fails at once: <code>no flatfs_tool.py</code> (set <code>UOS_TEST_SRC_DIR</code>) | the CTest case must pass <code>UOS_TEST_SRC_DIR</code> in its <code>ENVIRONMENT</code> |
| <code>NO RESULT</code> although the test printed PASS | the runner read the log before it was complete; fixed in <code>run-host.sh</code> / <code>run-qemu.sh</code> (pipeline + <code>PIPESTATUS</code>) — update your clone |
| a hardware test never ends | it prints <code>RESULT</code> but does not stop: call <code>hw_result::ok()</code> / <code>fail()</code> |
| a hardware test hangs after the first print | the controlling thread starves behind busy workers: raise its priority |
| a native test hangs only with clang 16–18 release | the CPU id was read through a cached thread pointer — fixed in <code>posix-arch</code> ; see <code>docs/posix-arch-port.md</code> §2 |

14. Reference tables

14.1 Files, by layer

| file | layer | edit when |
|--|-------|---|
| tests/package.json | 1 | new configuration, new test action, new toolchain |
| tests/CMakeLists.txt | 2 | practically never |
| tests/cmake/tests-main.cmake | 2 | a new platform prefix / a new port |
| tests/cmake/global-definitions.cmake | 2 | an upstream-platform suite switch |
| tests/platforms/<p>/cmake/definitions.cmake | 2 | new platform, board id |
| tests/platforms/<p>/cmake/dependencies-folders.cmake | 2 | a suite added to that platform |
| tests/platforms/<p>/cmake/platform-library.cmake | 2 | flags, linker, port libraries |
| tests/platforms/<p>/CMakeLists.txt | 2 | new platform; new suite on an upstream-style platform |
| tests/sources/<suite>/... | 2 | new or changed harness suite |
| <port>/test/CMakeLists.txt | 3 | the builder itself (rare, affects every board) |
| <port>/test/boards//board.cmake , hw.sh , openocd.cfg | 3 | new board, probe, memory map |
| <port>/test//tests.cmake | 3 | per-test knobs |
| <port>/test//<app>/... | 3 | new or changed board test |
| test_smp1/run-{qemu, host, hw}.sh | 4 | timeouts, verdict parsing |

14.2 Environment variables

| variable | read by | meaning |
|------------------------------------|-------------------|---|
| UOS_QEMU_ONLY | run-qemu.sh | run only this app |
| UOS_RUN_ONLY | run-host.sh | run only this app |
| UOS_QEMU_SHIM , UOS_QEMU_LOAD_ADDR | run-qemu.sh | AArch32 on raspb3b : shim image and load address |
| UOS_TEST_SRC_DIR | run-qemu.sh | where the board's tests (and flatfs_tool.py) are |
| UOS_HW_CFG , UOS_HW_* | run-hw.sh | OpenOCD configuration and ISA facts |
| BUILD | the boards' hw.sh | the build folder holding test/<app>-hwd |
| OPENOCD | the boards' hw.sh | an explicit OpenOCD binary |

14.3 CMake cache variables

| variable | meaning |
|---------------|-------------------------------------|
| PLATFORM_NAME | the platform (set by package.json) |

| variable | meaning |
|--|---|
| <code>CMAKE_BUILD_TYPE</code> | <code>Debug</code> , <code>Release</code> , <code>MinSizeRel</code> (set by <code>package.json</code>) |
| <code>CMAKE_TOOLCHAIN_FILE</code> | from <code>build-helper/cmake/toolchains/<toolchainFileName></code> |
| <code>BOARD</code> | the port's board id (set by the platform's <code>definitions.cmake</code>) |
| <code>UOS_SMP_DIR</code> , <code>UOS_AARCH32_DIR</code> , <code>UOS_AARCH64_DIR</code> ,
<code>UOS_CORTEXM_DIR</code> , <code>UOS_POSIX_ARCH_DIR</code> | the repositories in use (§6.3) |
| <code>UOS_DEBUG_BOOT</code> | cortexm: compile the <code>hwd</code> images with early-boot markers |

14.4 Related documents

- [README-DEVELOPER.md](#) — the upstream quick start this guide follows.
- [TESTS-CATALOG.md](#) — every platform, test and probe.
- [STEPS.md](#) — step-by-step install, build and run.
- [TESTS-XPack-SYSTEM.md](#) — the xPacK system in depth.
- [../posix-arch-port.md](#) — the native port, including the verdict and timing traps.